Databricks Streaming and Lakeflow Declarative Pipelines



Databricks Academy

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Agenda

1. Introduction to Streaming	Lecture	Demo	Lab
Streaming Data Concepts	✓		
Introduction to Structured Streaming	✓		
Reading from a Streaming Query		✓	
Streaming from Delta Lake	✓		
Streaming Query			✓
2. Aggregations, Time Windows, Watermarks	**Lecture**	**Demo**	**Lab**
Aggregations, Time Windows, Watermarks	✓		
Event Time + Aggregations Over Time Windows	✓		
Stream Aggregations			✓
Windowed Aggregation with Watermark		✓	
3. Streaming Joins (Optional)	**Lecture**	**Demo**	**Lab**
Streaming Joins (Optional)	✓		
Stream/Stream Joins (Optional)		✓	



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

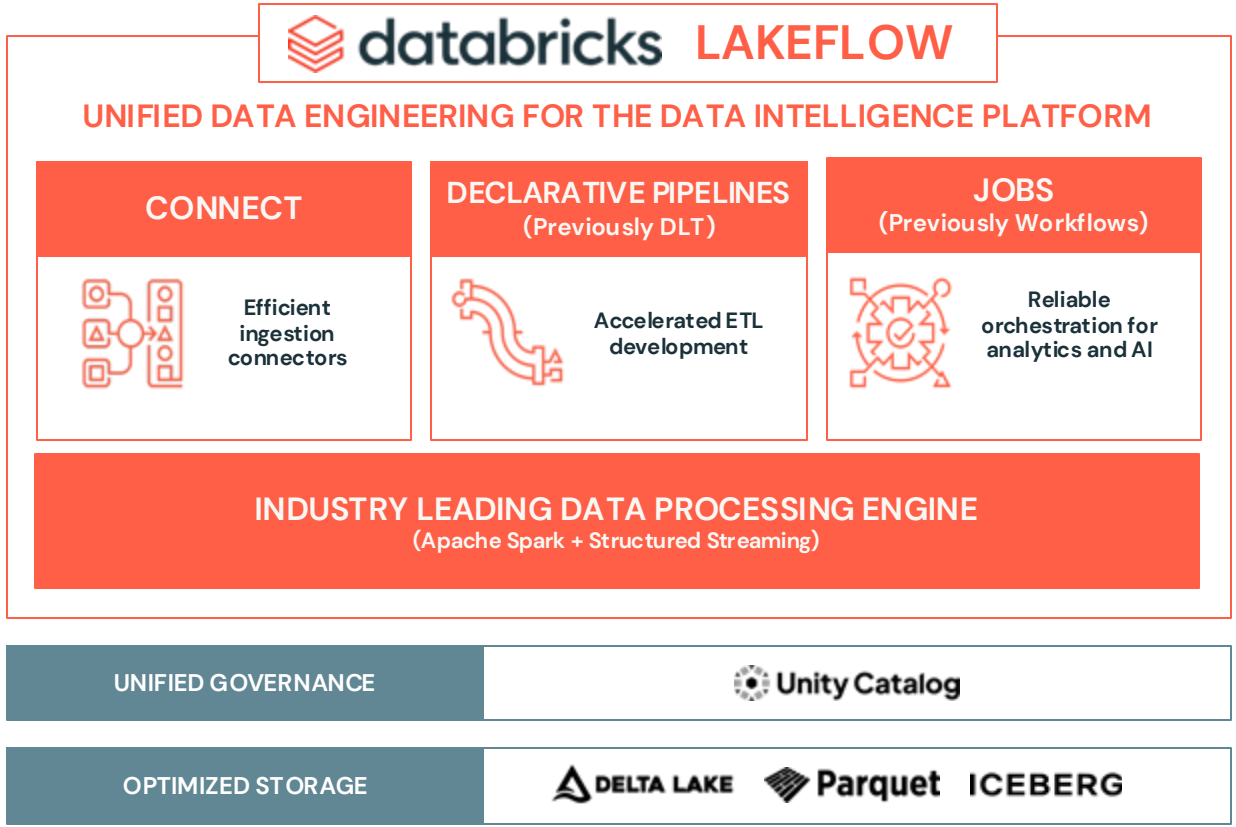
Agenda

4. Streaming ETL Patterns with Lakeflow Declarative Pipelines	Lecture	Demo	Lab
Data Ingestion Patterns	✓		
Auto Load to Bronze		✓	
Stream from Multiplex Bronze		✓	
Data Quality Enforcement	✓	✓	
Streaming ETL			✓



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

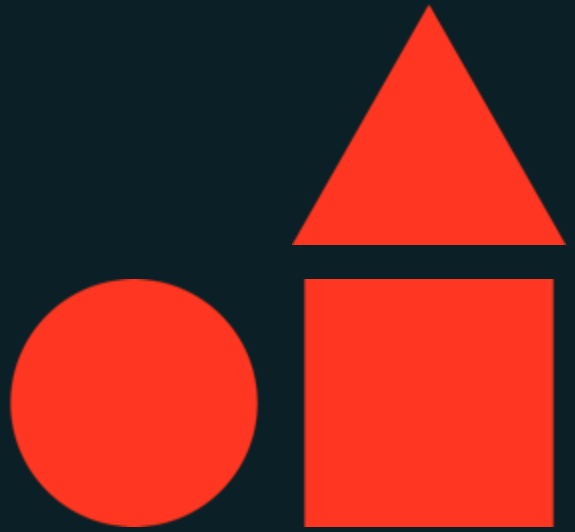
Data Engineering in Databricks



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming



Databricks Streaming and Lakeflow Declarative Pipelines

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LECTURE

Streaming Data Concepts



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Stream Processing

Why is stream processing getting popular?

Data Velocity & Volumes	Rising data velocity & volumes requires continuous, incremental processing - cannot process all data in one batch on a schedule
Real-time analytics	Businesses demand access to fresh data for actionable insights and faster, better business decisions
Operational applications	Critical applications need real-time data for effective, instantaneous response



The vast majority of the data in the world is streaming data!



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Stream Processing Use Cases

Stream processing is a key component of big data applications across all industries



Notifications



Real-time reporting



Incremental ETL



Update data to serve in real-time



Real-time decision making



Online ML

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Advantages of Stream Processing

Why use streaming (vs. batch) ?



Incremental processing of new data is natural and intuitive



Better fault-tolerance upon restarts



Unlocks low latency SLAs for time sensitive workloads



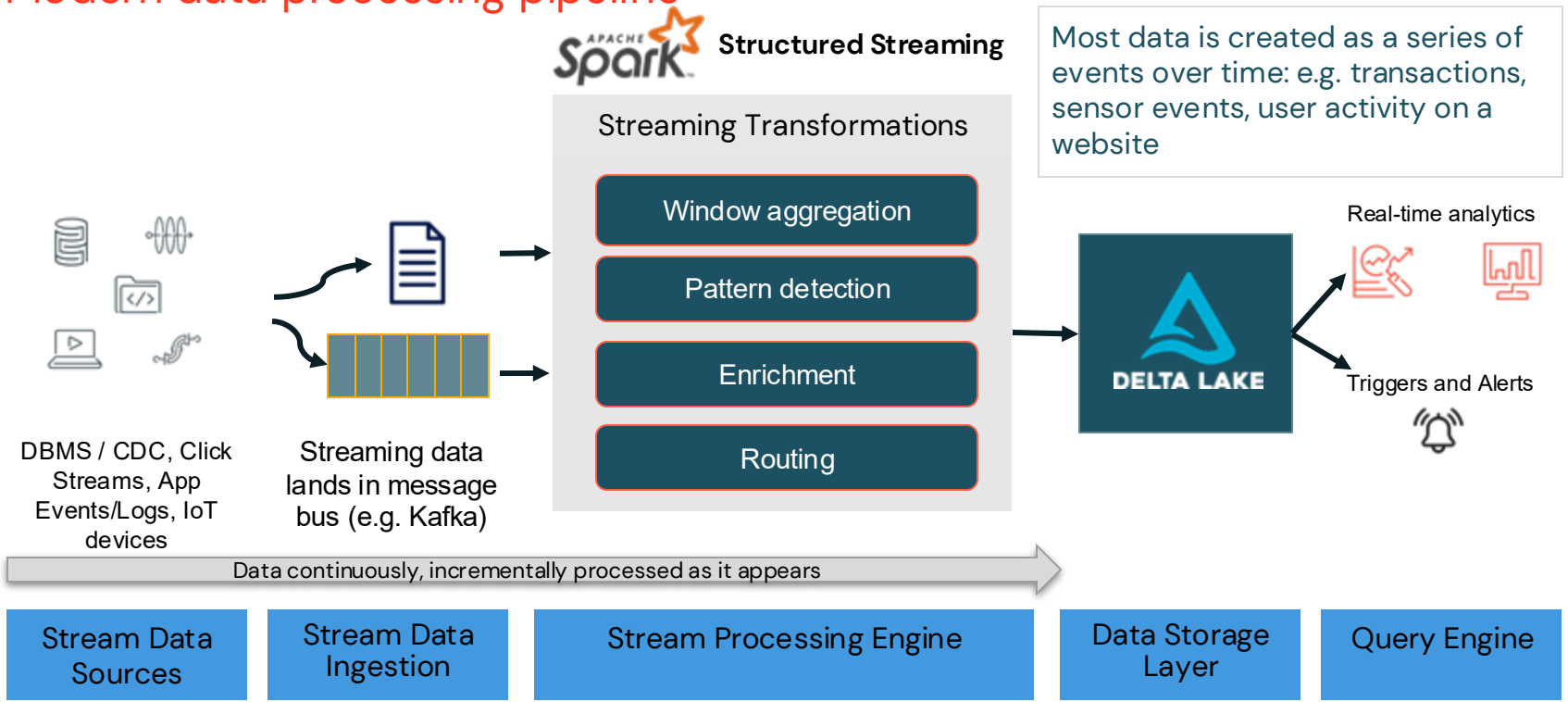
Higher compute utilization and scalability through continuous and incremental processing



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Stream Processing Architecture

Modern data processing pipeline



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Challenges of Stream Processing

Stream processing is not easy

- Processing each event exactly once despite machine failures
- Processing out-of-order data based on application timestamps (also called event time)
- Maintaining large amounts of state
- Handling load imbalance and stragglers
- Determining how to update output sinks as new events arrive
- Writing data transactionally to output systems



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LECTURE

Introduction to Structured Streaming



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

What is Structured Streaming

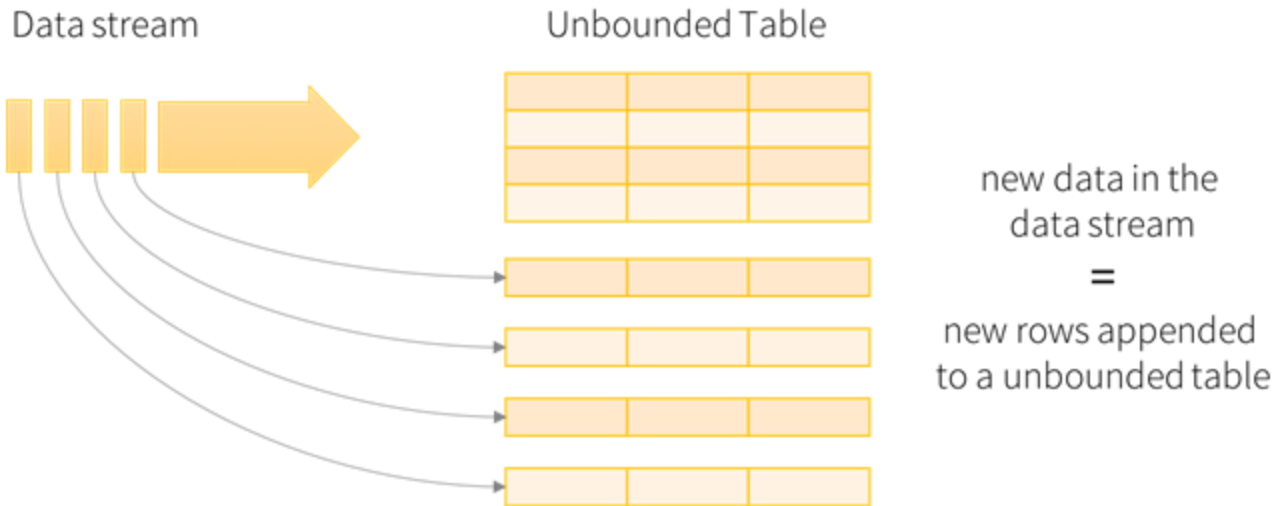
Apache Spark structured streaming basics

- A scalable, fault-tolerant **stream processing framework** built on Spark SQL engine.
- Uses **existing structured APIs** (DataFrames, SQL Engine) and provides similar API as batch processing API.
- Includes **stream specific features**; end-to-end, exactly-once processing, fault-tolerance etc.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Streams as Unbounded Tables

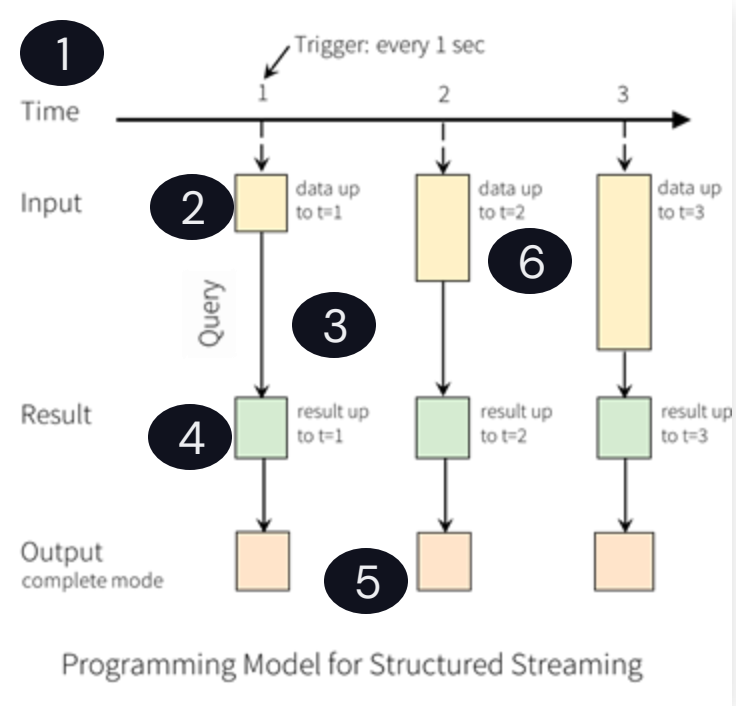


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

How Structured Streaming Works

Execution mode

1. Timeline
2. An **input table** is defined by configuring a streaming read against **source**.
3. A **query** is defined against the input table.
4. This logical query on the input table generates the **results table**.
5. The **output** of a streaming pipeline will persist updates to the results table by writing to an external **sink**.
6. New rows are appended to the input table for each **trigger interval**.



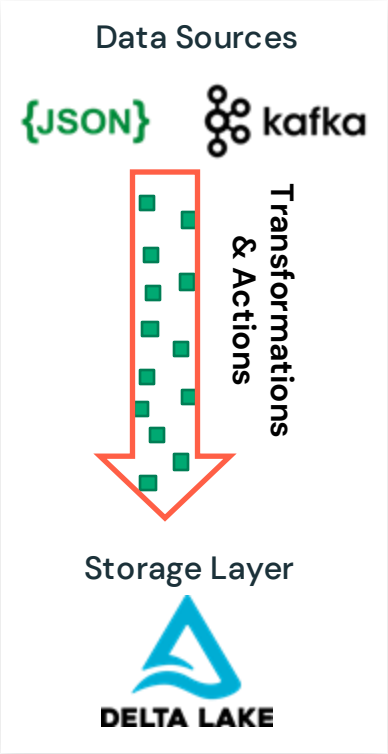
Programming Model for Structured Streaming

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured streaming core concepts

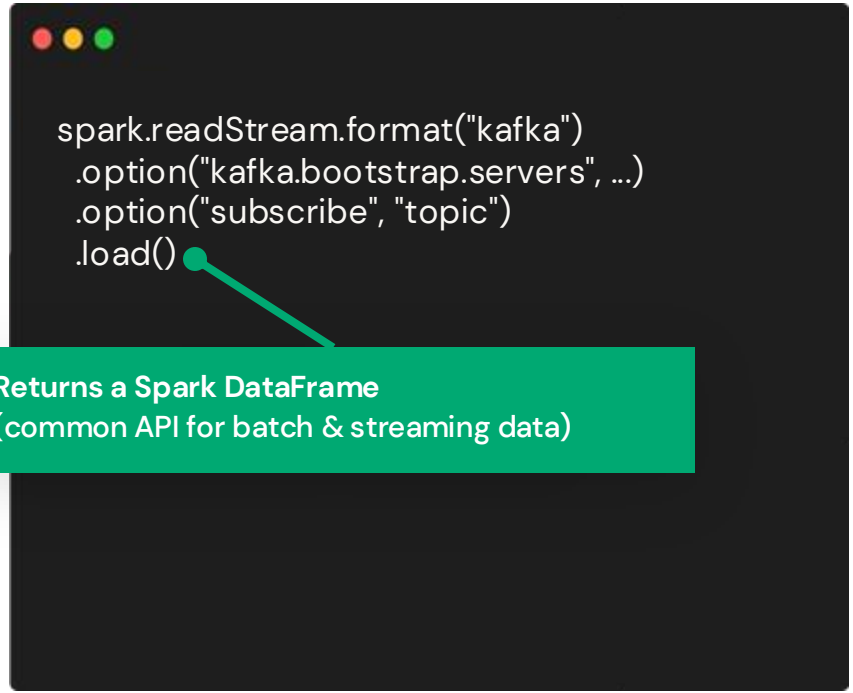
- Example:
  - Read JSON data from Kafka
  - Parse nested JSON
  - Store in structured Delta Lake table
- Core concepts:
  - Input sources
  - Transformations
  - Sinks
  - Triggers



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured streaming core concepts



Source:

- Specify where to read data from
- OS Spark supports Kafka and file sources
- Databricks runtimes include connector libraries supporting Delta, Event Hubs, and Kinesis

```
spark.readStream.format(<source>)
.option(<>,<>)...
.load()
```

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", ...)
  .option("subscribe", "topic")
  .load()
  .selectExpr("cast (value as string) as json")
  .select(from_json("json", schema).as("data"))
```

Transformations:

- 100s of built-in, optimized SQL functions like from_json
- In this example, cast bytes from Kafka records to a string, parse it as JSON, and generate nested columns



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", ...)
  .option("subscribe", "topic")
  .load()
  .selectExpr("cast (value as string) as json")
  .select(from_json("json", schema).as("data"))
  .writeStream
  .format("delta")
  .option("path", "/deltaTable/")
```

Sink: Write transformed output to external storage systems

Databricks runtimes include connector library supporting Delta

OS Spark supports:

- Files and Kafka for production
- Console and memory for development and debugging
- foreachBatch to execute arbitrary code with the output data



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured streaming core concepts

```
spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", ...)
  .option("subscribe", "topic")
  .load()
  .selectExpr("cast (value as string) as json")
  .select(from_json("json", schema).as("data"))
  .writeStream
  .format("delta")
  .option("path", "/deltaTable/")
  .trigger("1 minute")
  .option("checkpointLocation", "...")
  .start()
```

- **Checkpoint location:** For tracking the progress of the query
- **Output Mode:** Defines how the data is written to the sink; Equivalent to “save” mode on static DataFrames
- **Trigger:** Defines how frequently the input table is checked for new data; Each time a trigger fires, Sparks check for new data and updates the results

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

DataFrame as a Unifying API

Batch

```
spark
  .read
  .table("source_name")
  .withColumn("...")
  .select(...)
  .write
[ .outputMode(...) ]
  .saveAsTable("sink_table")
```

Streaming

```
spark
  .readStream
[ .withWatermark(...) ]
  .table("source_name")
  .withColumn("...")
  .select(...)
  .writeStream
[ .trigger(...) ]
[ .queryName(...) ]
  .option("checkpointLocation", ...)
  .toTable("sink_table")
```



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured streaming core concepts

Trigger Types:

Fixed interval micro batch	.trigger(processingTime = "2 minutes")	Micro-batch processing kicked off at the *user-specified interval*
Triggered One-time micro batch	.trigger(once=True)	DEPRECATED - Process all of the available data as a *single micro-batch* and then automatically stop the query
Triggered One-time micro batches	.trigger(availableNow=True)	Process all of the available data as *multiple micro-batches* and then automatically stop the query
Continuous Processing	.trigger(continuous= "2 seconds")	EXPERIMENTAL - Long-running tasks that *continuously* read, process, and write data as soon events are available, with checkpoints at the specified frequency
Default		Databricks: 500ms fixed interval OS Apache Spark: Process each microbatch as soon as the previous has been processed



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Anatomy of a Streaming Query

Structured Streaming Core Concepts

Output Modes:

Complete	• The entire updated Result Table is written to the sink. • The individual sink implementation decides how to handle writing the entire table.
Append	Only the new rows appended to the Result Table since the last trigger are written to the sink.
Update	Only new rows and the rows in the Result Table that were updated since the last trigger will be outputted to the sink.

Note: The output modes supported depends on the type of transformations and sinks used by the streaming query. Refer to the the Structured Streaming Programming Guide for details.



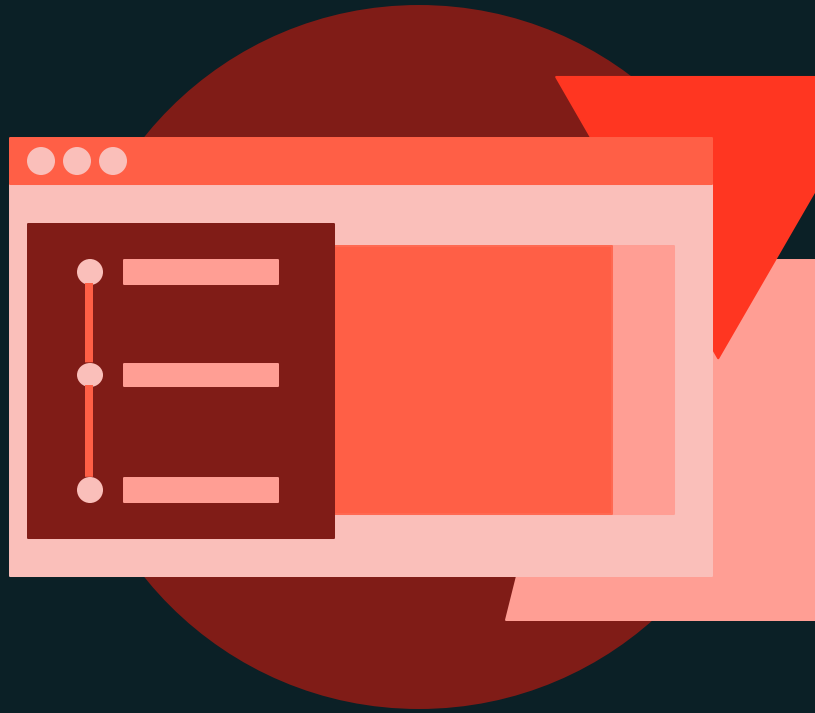
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

DEMONSTRATION

Reading from a Streaming Query



Notebook: SDLT 1 - Incremental Processing with Spark Structured Streaming/SDLT 1.1 - Reading from a Streaming Query

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LECTURE

Streaming from Delta Lake



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Streaming from Delta Lake

Using a Delta table as a streaming source

- Each committed version represents new data to stream. Delta Lake transactions logs identify the version's new data files
- Structured Streaming assumes append-only sources. Any non-append changes to a Delta table causes queries streaming from that table to throw exceptions.
  - Set `delta.appendOnly = true` to prevent non-append modifications to a table.
  - Use Delta Lake change data feed to propagate arbitrary change events to downstream consumers (discussed later in this course).



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Streaming from Delta Lake

Using a Delta table as a streaming source

- You can limit the input rate for micro-batches by setting DataStreamReader options:
  - maxFilesPerTrigger: Maximum files read per micro-batch (default 1,000)
  - maxBytesPerTrigger: Soft limit to amount of data read per micro-batch (no default)
  - Note: Lakeflow Declarative pipelines auto-tune options for rate limiting, so you should avoid setting these options explicitly for your pipelines.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Streaming to Delta Lake

Using a Delta table as a streaming sink

- Each micro-batch written to the Delta table is committed as a new version.
- Delta Lake supports both append and complete output modes.
  - Append is most common.
  - Complete replaces the entire table with each micro-batch. It can be used for streaming queries that perform arbitrary aggregations on streaming data.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LAB EXERCISE

Streaming Query Lab



Notebook: SDLT 1 - Incremental Processing with Spark Structured Streaming/SDLT 1.2L - Streaming Query Lab

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Aggregations, Time Windows, Watermarks



Databricks Streaming and Lakeflow Declarative Pipelines

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LECTURE

Aggregations, Time Windows, Watermarks



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Types of Stream Processing

Stateless vs. Stateful processing

- **Stateless**
  - Typically trivial transformations. The way records are handled do not depend on previously seen records.
  - Example: Data Ingest (map-only), simple dimensional joins
- **Stateful**
  - Previously seen records can influence new records
  - Example: Aggregations over time, Fraud/Anomaly Detection

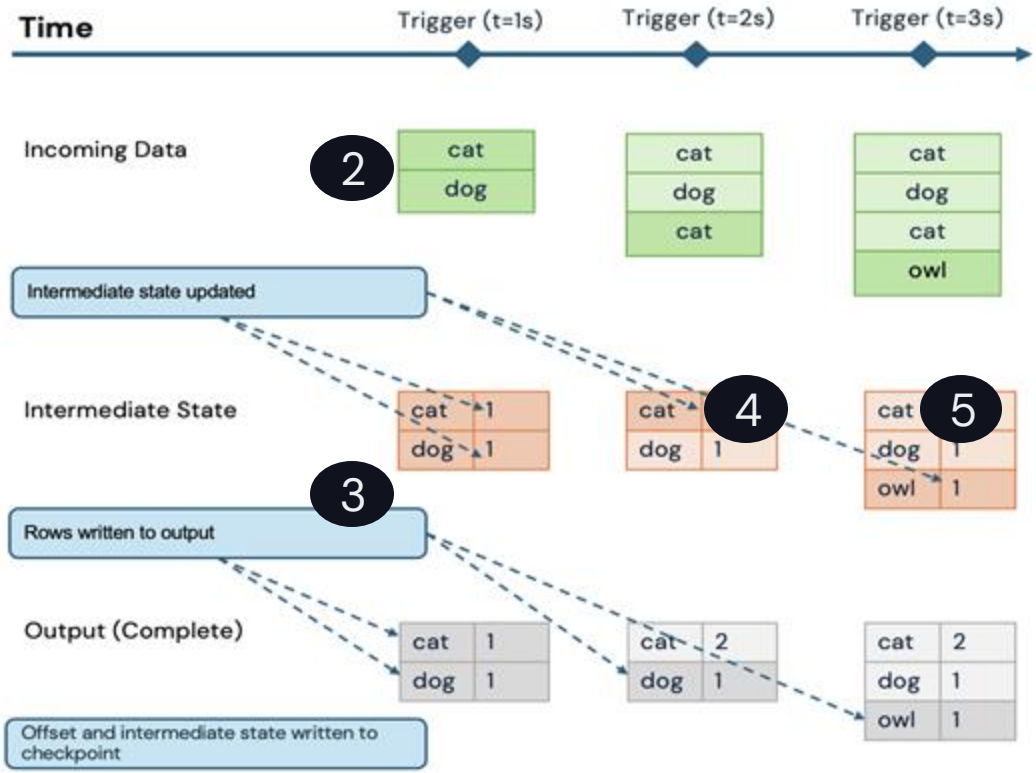


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Stateful Stream

```
spark
.readStream
.<source info>
.groupBy(animal)
.count()
.writeStream
.mode("complete")
.<sink info>
.trigger("1s")
.start()
```

1



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LECTURE

Event Time + Aggregations over Time Windows



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Reasoning About Time

Event time vs. Processing time

Event time vs. processing time

- **Event Time:** time at which the event (record in the data) actually occurred.
- **Processing time:** time at which a record is actually processed.
- Important in every use case processing unbounded data in whatever order (otherwise no guarantee on correctness)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Time Based Windows

Tumbling window vs. Sliding window

Tumbling Window

- **No window overlap**
- Any given event gets aggregated into **only one** window group (e.g. 1:00–2:00 am, 2:00–3:00 am, 3:00-4:00 am, …)

Sliding Window

- **Windows overlap**
- Any given event gets aggregated into **multiple window** groups (e.g. 1:00-2:00 am, 1:30–2:30 am, 2:00–3:00 am, …)

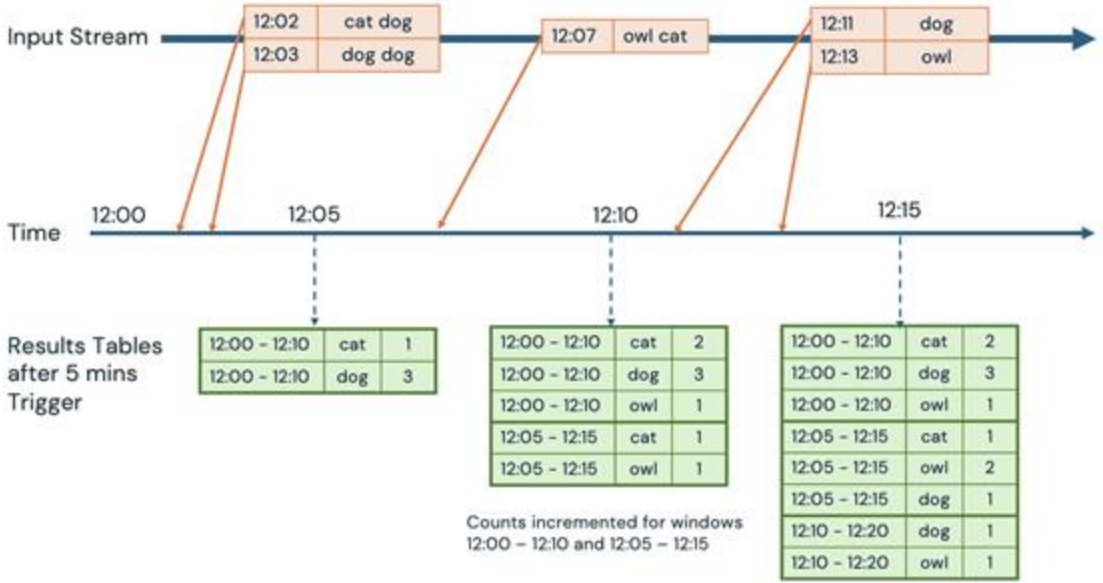


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Time Based Windows

Sliding window example

```
windowedDF =
 (
    eventsDF
    .groupBy(window("eventTime",
       "10 minutes",
       "5 minutes"))
    .count()
    .writeStream
    .trigger(processingTime="5 minutes")
 )
```

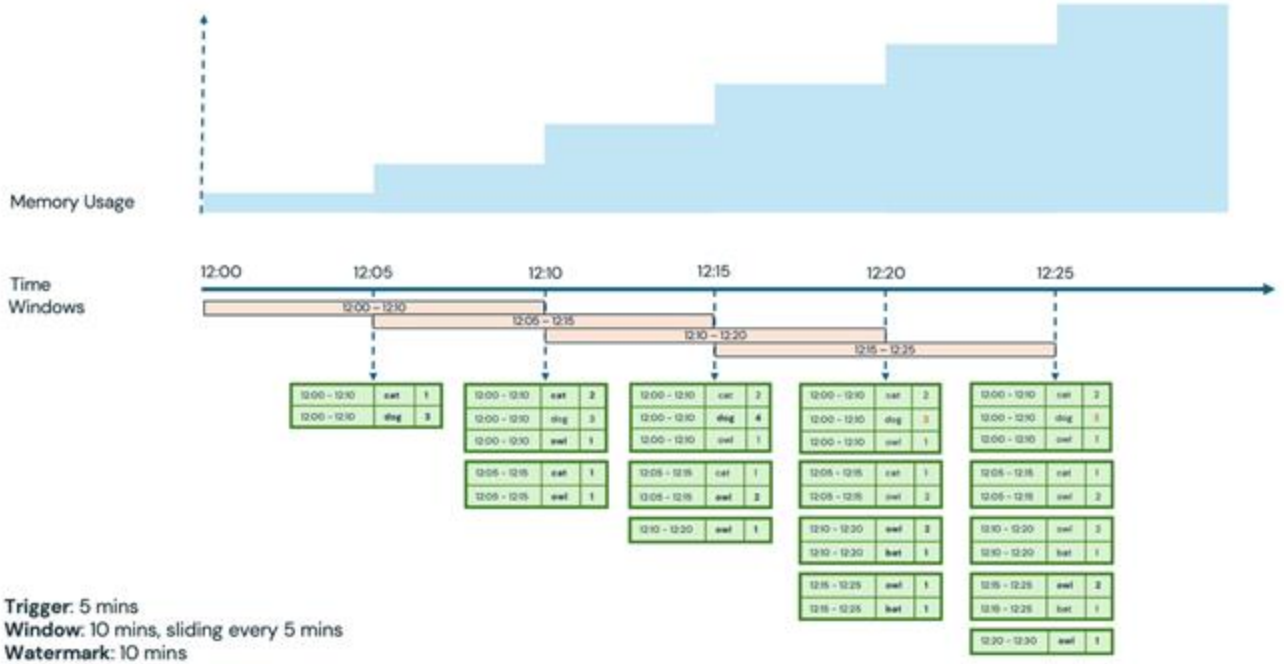


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Challenges: Memory Pressure

Long running query example

- Size of intermediate state will keep increase over time.
- Default behaviour is to maintain state in executor memory and DBFS.
- GC pauses will take longer and longer. Failure is inevitable.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Solution 1: RocksDB as external state store

One technique to reduce memory usage on the cluster is to save intermediate state off-heap.

```
spark
spark.conf.set(
  "spark.sql.streaming.stateStore.providerClass",
  "com.databricks.sql.streaming.state.RocksDBStateStoreProvider")
```

References:
Databricks Blog on RocksDB
Structured Streaming Programming Guide

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Solution 2: Watermarking / Late Threshold

Capture “Event Time” on incoming data and set Watermarks.

Purpose:

1. Reclaim memory by purging old state
2. Set expectations on “how late is too late?”

   Event Time < (Max event time seen by the engine - late threshold)

```
Spark
.readStream
.withWatermark(“event_timestamp”, “10 minutes”)
.<source info>
```

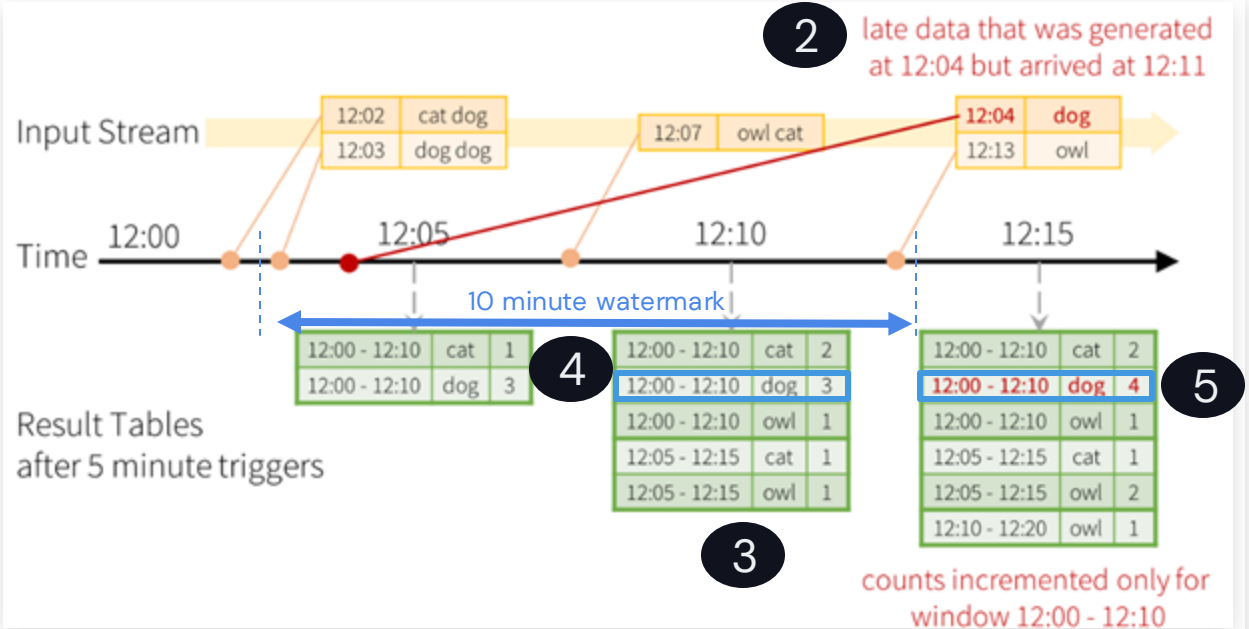
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Handling Late Arrival Events with Watermark

Delayed data will be processed if within Watermark threshold - Guaranteed!

1 Configuration:

1. **Trigger**: 5 min
2. **Window**: 10 mins
3. **Sliding**: 5 min
4. **Watermark:** 10 mins

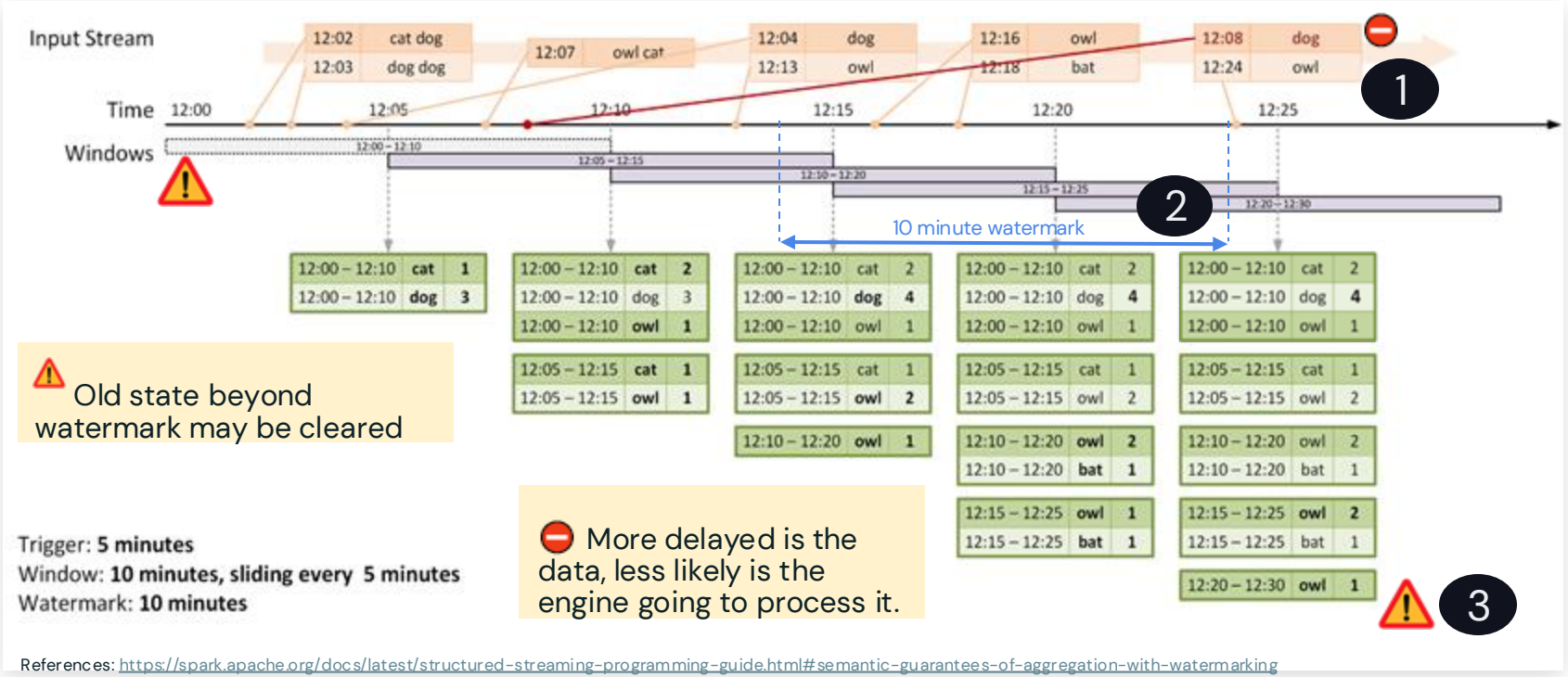


References: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#semantic-guarantees-of-aggregation-with-watermarking

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Control size of state

Late data MAY be dropped



References: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#semantic-guarantees-of-aggregation-with-watermarking

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LECTURE

Trigger Types and Output Modes



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Trigger Types

When to stream data in

Default	Process each micro-batch as soon as the previous one has been processed (no coding required)
Fixed Interval	Micro-batch processing kicked off at the user-specified interval: `.trigger(processingTime = '1 second'))`
One-time	Process all available data as a single micro-batch and then automatically stop the query: `.trigger(once = True)`
availableNow	Like **Trigger Once,** available data processed before query stops, but in multiple batches instead of one: `.trigger(availableNow = True)`
Continuous Processing	Long-running tasks that continuously read, process, and write data as soon events are available: `.trigger(continuous = '1 second')`



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Output Modes

Input

```
hello world I say hello
hello goodbye
```

What to write

APPEND

Add new records only (*default*)

value
hello
say
I
say
hello

value
hello
goodbye

UPDATE

Update changed records in place

value	count
hello	2
say	1
world	1
I	1

value	count
hello	2
goodbye	1

COMPLETE

Rewrite full output

value	count
hello	2
say	1
world	1
I	1

value	count
hello	3
say	1
world	1
I	1
goodbye	1

Only rows updated since last trigger written. Different from **Complete** Mode since **Update** Mode outputs only changed rows since last trigger. If query doesn't contain aggregations, **Update** same as **Append** mode

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

LAB EXERCISE

Stream Aggregations Lab



Notebook: SDLT 1 - Incremental Processing with Spark Structured Streaming/SDLT 1.3L - Stream Aggregations Lab

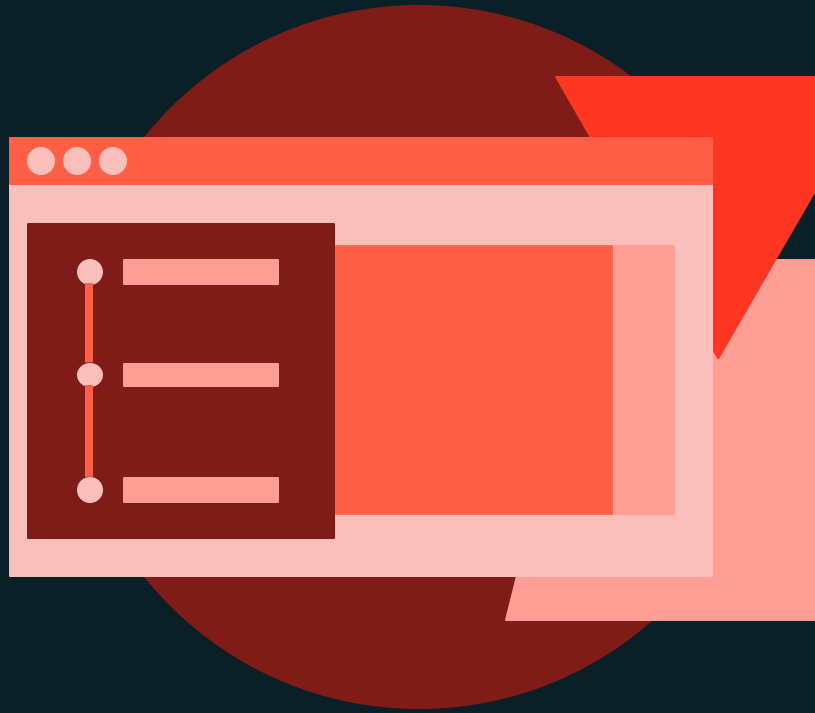
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Introduction to Streaming

DEMONSTRATION

Windowed Aggregation with Watermark

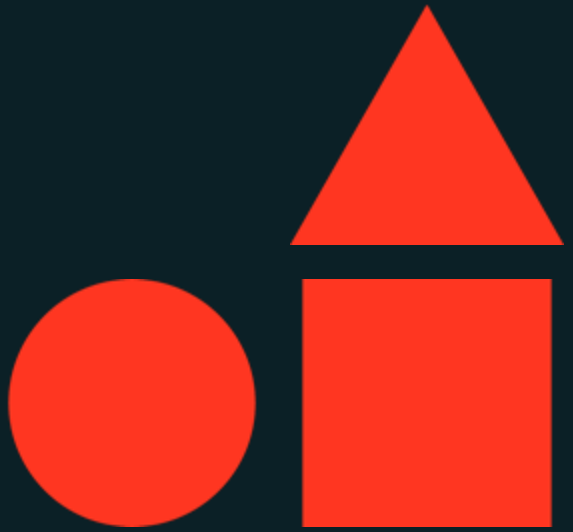


Notebook: SDLT 1 - Incremental Processing with Spark Structured Streaming/SDLT 1.4 - Windowed Aggregation with Watermark

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming Joins (Optional)



Databricks Streaming and Lakeflow Declarative Pipelines

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming Joins

LECTURE

Streaming Joins (Optional)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Join Operations

- Structured Streaming supports joining a streaming Dataset/DataFrame with a static Dataset/DataFrame as well as another streaming Dataset/DataFrame.
- The result of the streaming join is generated incrementally, similar to the results of streaming aggregations in the previous examples.
- In all the supported join types, the result of the join with a streaming Dataset/DataFrame will be the exactly the same as if it was with a static Dataset/DataFrame containing the same data in the stream.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Managing Memory Pressure

As the stream runs, the size of streaming state will keep growing indefinitely as all past input must be saved as any new input can match with any input from the past.

Avoid Unbounded State by:

1. Define Watermark delays on both inputs
2. Add **Event-Time Constraints** on join condition

   e.g. `JOIN ON leftTime BETWEEN rightTime AND rightTime + INTERVAL 1 HOUR`

Reference: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#stream-stream-joins



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Streaming Joins Supported

Left Side ↓	Right Side →	
	Static	**Stream**
Static	✅ All join types supported - Stateless (This is not on streaming join even though it can be present in a streaming query)	✅ Inner Join - Stateless ✅ Right Outer - Stateless ⛔ All Other joins
Stream	✅ Inner Join - Stateless ✅ Left Outer - Stateless ✅ Left Semi - Stateless ⛔ All Other joins	✅ Inner Join - Stateful. Watermarks Optional ✅ Left Outer - Stateful. Watermark mandatory on right ✅ Left Semi - Stateful. Watermark mandatory on right ✅ Right Outer - Stateful. Watermark mandatory on left ✅ Full Outer - Stateful. Watermark mandatory on at least side (Specify watermark on both sides + event time constraints for full state cleanup)

Reference: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#support-matrix-for-joins-in-streaming-queries

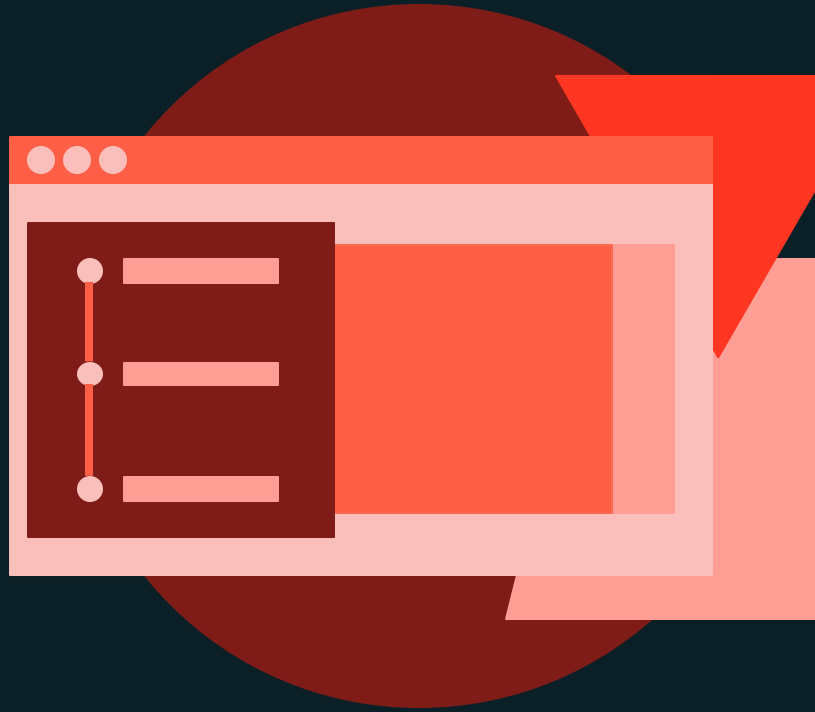
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming Joins

DEMONSTRATION

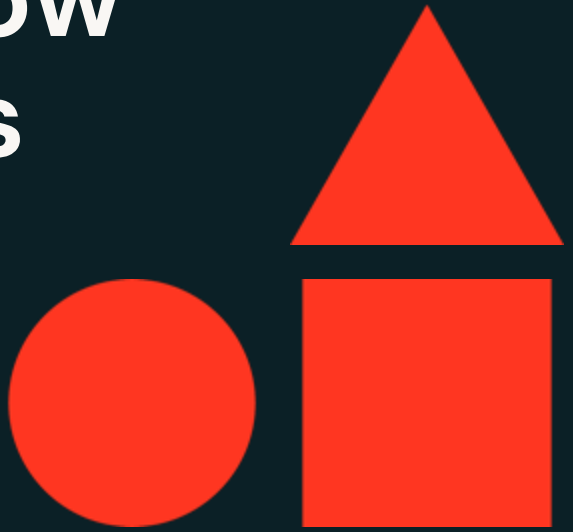
Stream/Stream Joins (Optional)



Notebook: SDLT 1 - Incremental Processing with Spark Structured Streaming/SDLT 1.5 - Optional - Stream-Stream Joins

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Streaming ETL Patterns with Lakeflow Declarative Pipelines

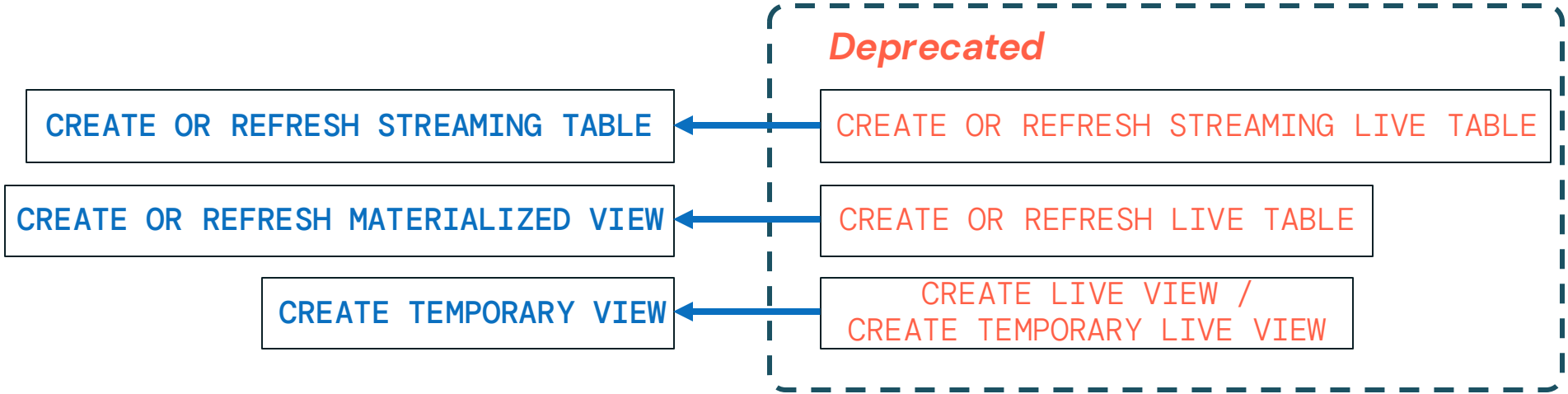


Databricks Streaming and Lakeflow Declarative Pipelines

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Lakeflow Declarative Pipelines Terminology

Existing users of DLT will notice that we've evolved the names



The **semantics remain the same**, and **we'll support the old syntax** for compatibility.
Our goal is to simplify the syntax and match other systems

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming ETL Patterns with Lakeflow Declarative Pipelines

LECTURE

Data Ingestion Patterns



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Why Do We Need These Patterns?

Limitations at Data Ingestion Stage

- Streaming sources like Kinesis, Kafka and EventHubs only retain data for a **limited** amount of time
- **Need for retention** – full history of data
  - Reprocessing raw data
  - Perform GDPR and compliance tasks
  - Recover data
- Need for a simple, **maintainable and scalable** architecture
- Keeping full history in the streaming source is **expensive**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern 1: Use Delta for Infinite Retention

Delta provides cheap, elastic and governable storage for transient sources




```
CREATE STREAMING
TABLE AS …
```

```
TBLPROPERTIES (
  pipelines.reset.allowed=false
)
```

bronze



```
CREATE
MATERIALIZED VIEW AS …
```

Use a **short retention** period to avoid compliance risks and reduce costs

Avoid complex transformations that could have bugs or drop important data

Retain history
Easy to perform GDPR and other compliance tasks

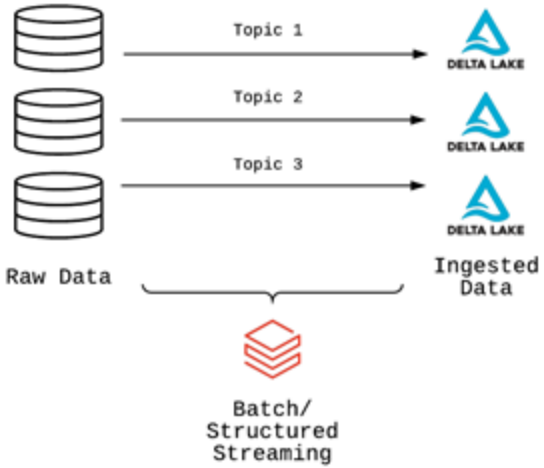
pipelines.reset.allowed=false ensures that downstream computation **can be full-refreshed without losing data**

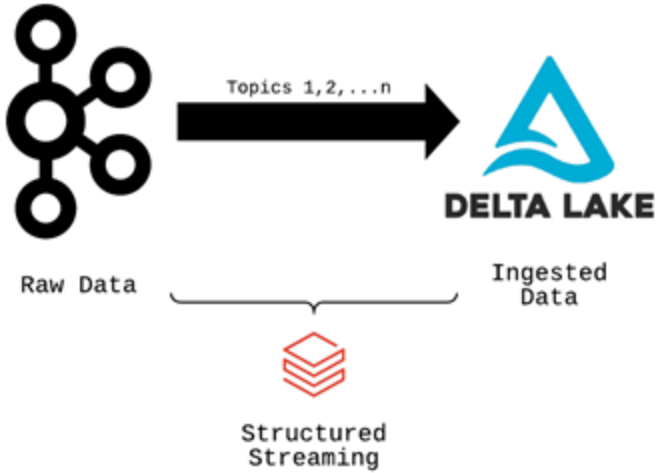
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern 2: Multiplex Ingestion

Multiplexing is used when a set of independent streams all share the same source

- First, let's discuss Simplex vs. Multiplex patterns





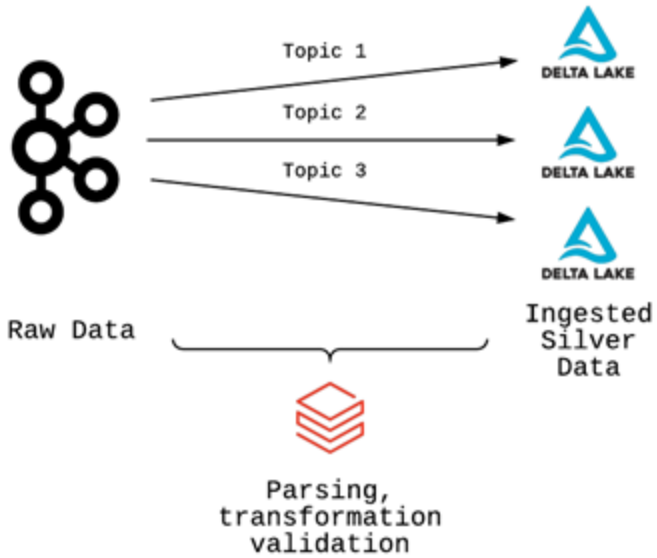
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern 2: Multiplex Ingestion

Anti-Pattern: Using Kafka as Bronze Table

Don’t use Kafka as Bronze Table:

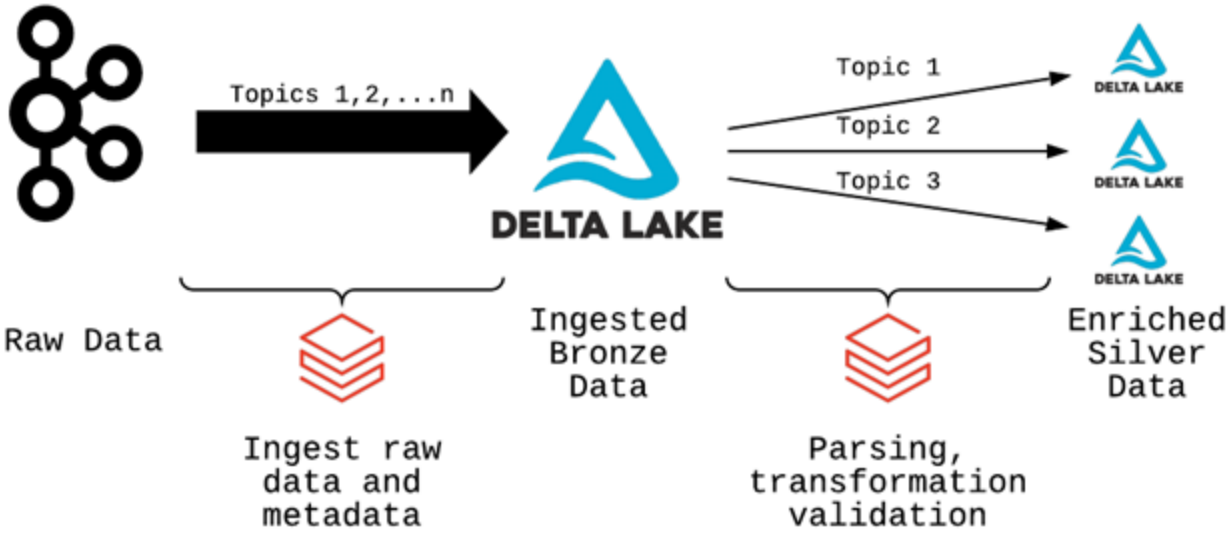
- Data retention limited by Kafka; expensive to keep full history
- All processing happens on ingest
- If stream gets too far behind, data is lost
- Cannot recover data (no history to replay)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern 2: Multiplex Ingestion Pattern

Multiplexing is used when a set of independent streams all share the same source

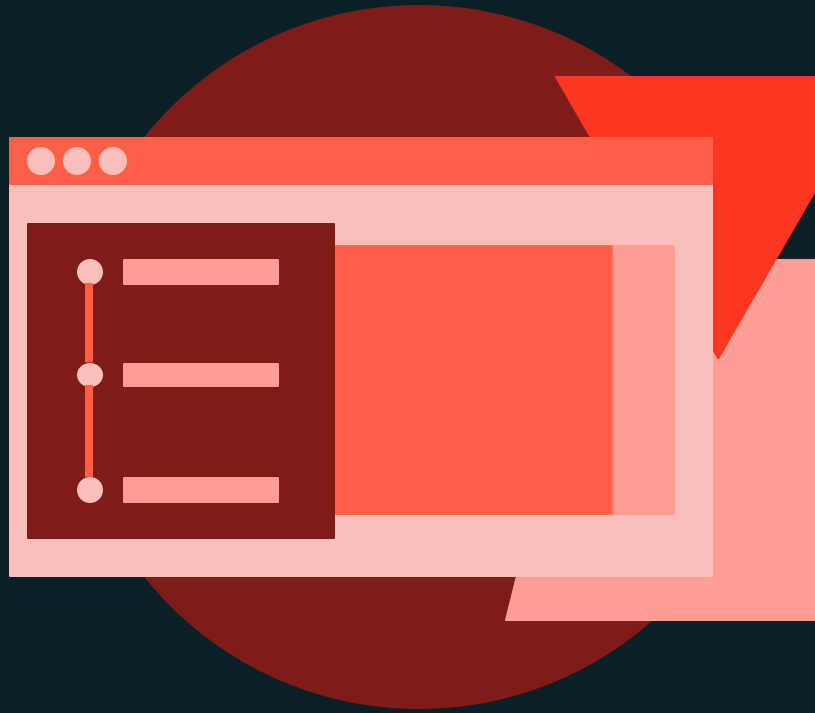


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming ETL Patterns with Lakeflow Declarative Pipelines
DEMONSTRATION

Auto Load to Bronze



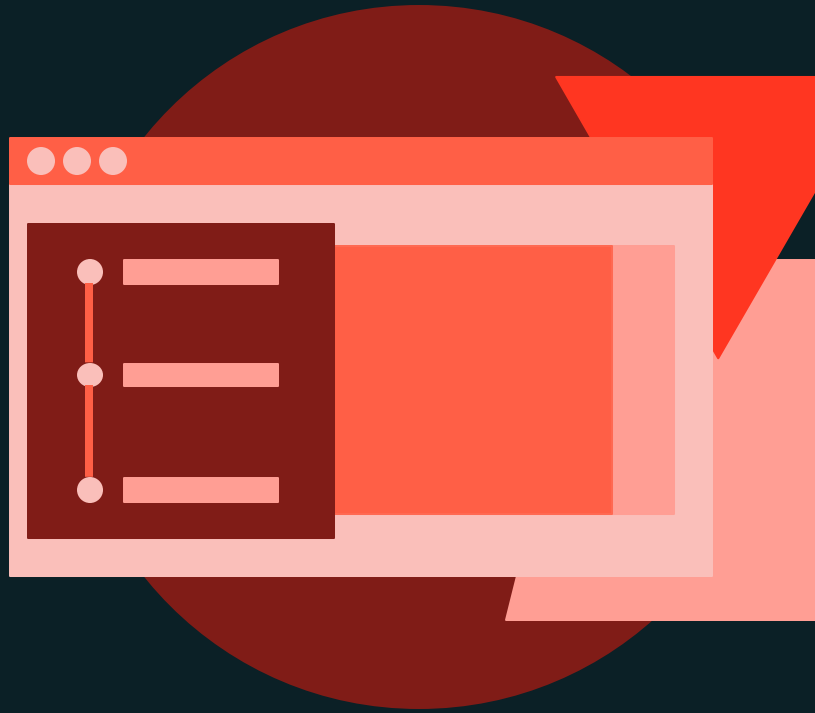
Notebook: SDLT 2 - Streaming ETL Patterns with Lakeflow Declarative Pipelines/SDLT 2.1 - Auto Load to Bronze

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming ETL Patterns with Lakeflow Declarative Pipelines
DEMONSTRATION

Stream from Multiplex Bronze



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming ETL Patterns with Lakeflow Declarative Pipelines

LECTURE

Data Quality Enforcement



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Silver Layer for Quality Enforcement

Silver Layer Objectives

- Validate data quality and schema
- Enrich and transform data
- Optimize data layout and storage for downstream queries
- Provide single source of truth for analytics



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Schema Enforcement & Evolution

- Enforcement prevents bad records from entering table
  - Mismatch in type or field name
- Evolution allows new fields to be added
  - Useful when schema changes in production/new fields added to nested data
  - Cannot use evolution to remove fields
  - All previous records will show newly added field as Null
    - For previously written records, the underlying file isn't modified.
    - The additional field is simply defined in the metadata and dynamically read as null



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Alternative Quality Check Approaches

- Add a “validation” field that captures any validation errors and a null value means validation passed.
- Quarantine data by filtering non-compliant data to alternate location
- Warn without failing by writing additional fields with constraint check results to Delta tables

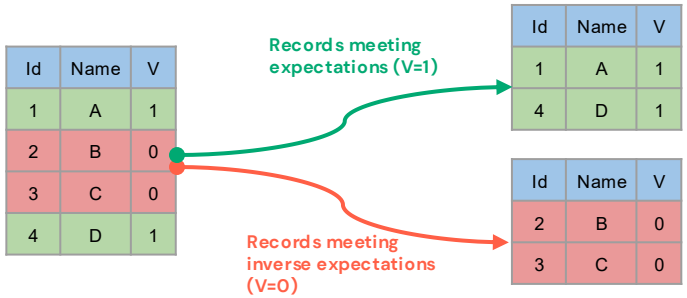


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern: Quarantine Invalid Records

What if we want to save the records that violate data quality constraints for analysis?

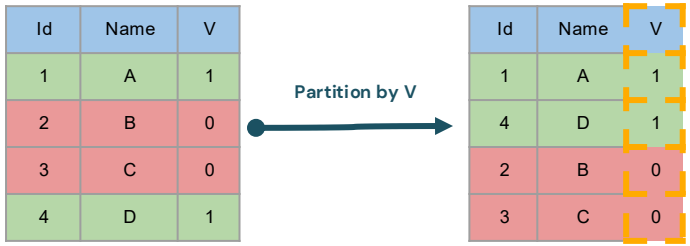
Method 1: Create Inverse Expectation Rules



Limitations:

- Processes the data twice

Method 2: Add a validation status column and use for partitioning



Limitations:

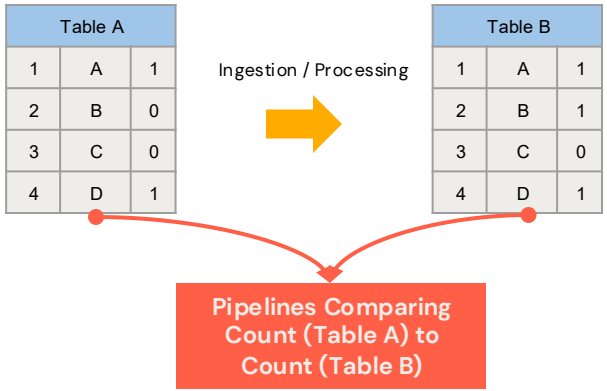
- Doesn't use expectations; data quality metrics are not available in the event logs or the pipelines UI.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern: Verify Data with Row Comparison

Validate row counts across tables to verify that data was processed successfully without dropping rows.

- Solution:
  - Add an additional table to your pipeline that defines an expectation to perform the comparison.
  - The results of this expectation appear in the event log and the Lakeflow Declarative pipelines UI.



1: Lakeflow Pipeline SQL

```
CREATE OR REFRESH MATERIALIZED VIEW count_verification(
  CONSTRAINT no_rows_dropped EXPECT (a_count == b_count)
)
AS
SELECT *
FROM
  (SELECT COUNT(*) AS a_count FROM tbla) a,
  (SELECT COUNT(*) AS b_count FROM tblb) b;
```

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Pattern: Define Tables for Adv. Validation

Perform advanced data validation with Pipeline expectations

- Complex data quality checks examples;
  - A derived table contains all records from the source table
  - Guaranteeing the equality of a numeric column across tables
- Solution:
  - Define a Lakeflow Declarative Pipeline using aggregate and join queries and use the results of those queries as part of your expectation checking.

```
CREATE OR REFRESH MATERIALIZED VIEW compare_tests (
  CONSTRAINT no_missing_records EXPECT (r.key IS NOT NULL)
)
AS
SELECT *
FROM validation_copy v
LEFT OUTER JOIN report r
  ON v.key = r.key;
```

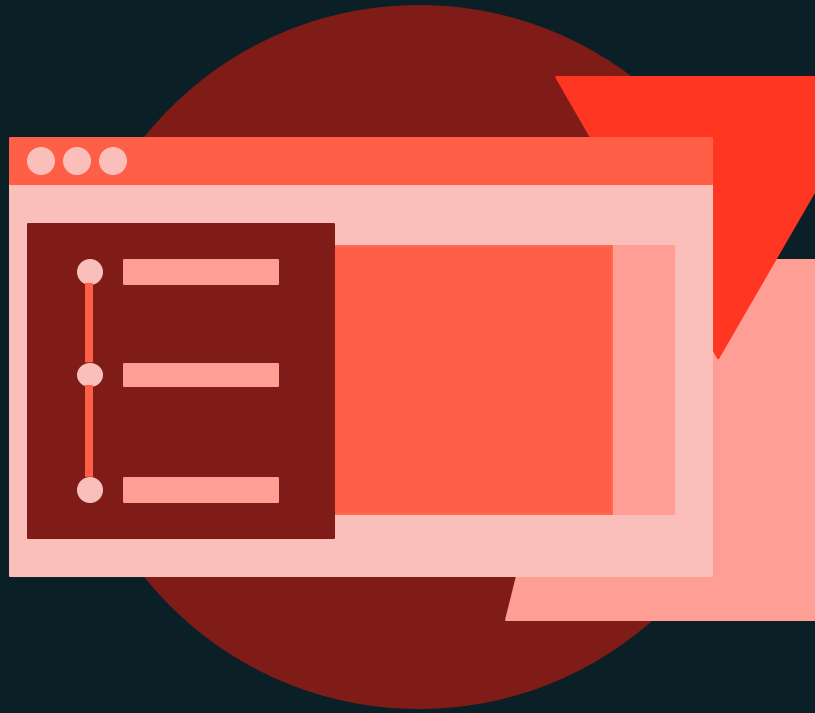


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming ETL Patterns with Lakeflow Declarative Pipelines
DEMONSTRATION

Data Quality Enforcement



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

databricks

Streaming ETL Patterns with Lakeflow Declarative Pipelines

LAB EXERCISE

Streaming ETL Lab



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Questions?



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.